Product Requirements Document

Dental Clinic Management System with AI-Powered Chatbot

Prepared by	Sneha Hibare
Document Type	Product Requirements Document (PRD)
Project	Dental Clinic Management System (DCMS)
Purpose	Internship project specification - built module-wise for engineering handoff
Version	1.0

1. Executive Summary

The Dental Clinic Management System (DCMS) is a full-stack web application that digitizes the end-to-end operations of a dental clinic: patient management, appointment scheduling, doctor workflows, treatment records, and billing. The system is enhanced with an AI-powered chatbot assistant that allows patients to interact in natural language to book, reschedule, or cancel appointments, ask clinic-related questions, and receive basic symptom-based department suggestions. The chatbot is built using LLM function calling, meaning the AI model interprets user intent and triggers real backend actions rather than only producing text.

This document breaks the system down module-by-module, with objectives, features, data requirements, and functional behaviour for each, so that each module can be built, reviewed, and tested independently before integration.

2. Problem Statement

Dental clinics commonly rely on manual registers, phone calls, and spreadsheets to manage patients, appointments, and billing. This leads to double-bookings, lost treatment history, delayed patient communication, and inefficient front-desk operations. DCMS solves this by centralising all clinic operations into a single system with a conversational AI interface that reduces the manual workload on staff and improves the patient experience.

3. Goals and Objectives

- Provide a single platform for patients, doctors, and admin/reception staff.
- Allow patients to self-serve appointment booking, rescheduling, and cancellation.
- Maintain accurate, centralised treatment history and prescriptions per patient.
- Automate billing generation linked to treatments.

- Provide an AI chatbot that can perform real actions (not just answer questions) via function calling.
- Ensure the chatbot operates within safe, well-defined guardrails (no diagnosis, no data leakage, in-scope only).
- Design the system to be modular, so each module can be developed and tested independently.

4. User Roles

Role	Description	Key Permissions
Patient	End user of the clinic	Register/login, book/reschedule/cancel appointments, chat with AI assistant, view own treatment history and bills
Doctor	Clinical staff	View daily schedule, view assigned patient history, add treatment notes and prescriptions
Admin / Receptionist	Clinic operations staff	Manage doctors, manage appointments across the clinic, manage billing, oversee all patient records

MODULE 1

Authentication & User Management

4.1 Objective

Securely manage identity and access for three distinct user roles, ensuring each role only accesses data and actions relevant to it.

4.2 Features

- Patient signup with name, phone number, email, date of birth, password.
- OTP-based or password-based login for patients.
- Separate login for doctors and admin/reception staff (created by admin, not self-registered).
- Role-based access control (RBAC): patient, doctor, admin.
- JWT-based session/token authentication for the backend API.
- Forgot password / reset password flow.
- Profile management (edit contact details, change password).

4.3 Data Requirements

Field	Type	Notes
user_id	UUID / INT	Primary key
name	String	Full name
email	String	Unique
phone	String	Unique, used for OTP login
password_hash	String	Encrypted, never stored in plain text
role	Enum	patient / doctor / admin
created_at	Timestamp	

4.4 Edge Cases

- Duplicate signup attempt with an already-registered phone/email.
- Expired or incorrect OTP entry.
- Login attempt for a deactivated account.

MODULE 2

Patient Management

5.1 Objective

Maintain a complete, accurate digital profile for every patient, including personal details and medical/dental history, accessible to the patient and to authorised clinical staff.

5.2 Features

- Patient profile with personal details, emergency contact, and known allergies.
- View past appointments and treatment history in one place.
- View and download prescriptions issued during past visits.
- Admin/doctor can search patients by name or phone number.

5.3 Data Requirements

Field	Type	Notes
patient_id	UUID / INT	Foreign key to users table
dob	Date	
gender	Enum	Optional
address	String	Optional
allergies	Text	Optional, free text
emergency_contact	String	Optional

MODULE 3 Doctor & Staff Management

6.1 Objective

Allow admin to manage the roster of doctors, their specialisations, and their availability, which directly feeds into the appointment booking logic.

6.2 Features

- Admin can add/edit/deactivate doctor profiles.
- Each doctor has a specialisation (e.g. General Dentist, Orthodontist, Endodontist, Pediatric Dentist).
- Each doctor sets or has an assigned weekly availability schedule (days + time slots).
- Doctor dashboard shows today's/upcoming appointments.

6.3 Data Requirements

Field	Type	Notes
doctor_id	UUID / INT	Foreign key to users table
specialization	String	
available_days	Array / JSON	e.g. Mon-Sat
available_slots	JSON	Start/end time per day
status	Enum	active / inactive

MODULE 4

Appointment Management

7.1 Objective

Core scheduling engine of the system - allows patients to book appointments and prevents conflicts such as double-booking a doctor's time slot.

7.2 Features

- Patient selects reason for visit or department, then sees relevant available doctors.
- Calendar-based slot picker showing only free slots for the selected doctor and date.
- Booking confirmation shown in-app and sent via SMS/email notification.
- Reschedule and cancel flows with reason capture.
- Automated reminder notification sent ahead of the appointment.
- Admin/reception view of the full day's schedule across all doctors.

7.3 Data Requirements

Field	Type	Notes
appointment_id	UUID / INT	Primary key
patient_id	FK	References patient
doctor_id	FK	References doctor
date	Date	
time_slot	Time	
reason	String	Symptom/purpose of visit
status	Enum	booked / completed / cancelled / no-show
created_via	Enum	app / chatbot / admin

7.4 Edge Cases

- Two patients attempting to book the same doctor/slot simultaneously (slot locking required).
- Patient tries to book a slot in the past.
- Doctor becomes unavailable after a slot is already booked (admin reassignment flow needed).
- Cancellation within a very short window of the appointment time.

MODULE 5

Treatment Records

8.1 Objective

Give doctors a way to document each visit, and give patients a permanent, accessible record of what treatment was performed.

8.2 Features

- Doctor adds notes, diagnosis, procedure performed, and prescription after each appointment.
- Record is linked to the specific appointment and visible in the patient's history.
- Follow-up date can be attached, which can trigger a future reminder.

8.3 Data Requirements

Field	Type	Notes
record_id	UUID / INT	Primary key
appointment_id	FK	References appointment
diagnosis	Text	
procedure	Text	
prescription	Text	
follow_up_date	Date	Optional

MODULE 6 Billing & Invoicing

9.1 Objective

Generate and track invoices for treatments performed, and allow patients to view their payment status.

9.2 Features

- Auto-generate a bill linked to a completed treatment/appointment.
- Admin can mark payment as paid/pending/partially paid.
- Patient can view billing history and outstanding dues.

9.3 Data Requirements

Field	Type	Notes
bill_id	UUID / INT	Primary key
appointment_id	FK	References appointment
amount	Decimal	
status	Enum	paid / pending / partial
payment_date	Date	Optional

MODULE 7

AI Chatbot Assistant

10.1 Objective

Provide a conversational, natural-language interface for patients that can perform real system actions (not just answer questions), using LLM function calling connected to the backend.

10.2 Core Capabilities

- Book, reschedule, and cancel appointments through conversation.
- Answer FAQs: clinic timing, location, services offered, doctor specialisations.
- Basic symptom-based triage: suggest which department/doctor to consult.
- Check status of an existing appointment.
- Hand off to a human/reception flow when the query is outside its scope.

10.3 How It Works (Architecture)

The chatbot uses an LLM with function calling. The user's message is sent to the LLM along with a system prompt (defining its role and boundaries) and a set of defined "tools" (functions) such as check_doctor_availability, book_appointment, cancel_appointment, get_clinic_info, and suggest_department. The LLM decides which function to call and with what parameters, the backend executes the real database operation, and the result is passed back to the LLM to generate a natural-language response for the user.

10.4 Chatbot Functions (Tools)

Function	Purpose	Key Inputs
check_doctor_availability	Check open slots for a doctor/date	doctor, date, time
book_appointment	Create a new appointment	patient_id, doctor, date, time, reason
cancel_appointment	Cancel an existing appointment	appointment_id
reschedule_appointment	Move an appointment to a new slot	appointment_id, new date/time
get_clinic_info	Return timing/location/services info	query_type
suggest_department	Map symptoms to a department/doctor	symptoms

10.5 Guardrails (Safety Boundaries)

- Scope control: chatbot only handles dental/clinic-related topics; politely declines unrelated topics.
- No medical diagnosis: it may suggest a relevant department but must direct the user to consult a doctor for any diagnosis.
- Data privacy: chatbot only accesses the logged-in patient's own data, never another patient's.
- Graceful fallback: unclear queries get a clarifying question instead of a wrong or made-up answer.
- Professional tone: no offensive, casual-inappropriate, or off-brand language.
- Action confirmation: chatbot confirms critical actions (booking/cancelling) with the user before finalising.

10.6 Example Conversation Flow

User: "I have tooth pain, what should I do?" → Chatbot asks 1-2 clarifying questions about the symptom → calls suggest_department → recommends General Dentist → offers to check availability and book directly within the same conversation.

MODULE 8

Admin / Reception Dashboard

11.1 Objective

Give clinic staff a centralised operational view and control over doctors, appointments, and billing across the whole clinic.

11.2 Features

- Full-day/week calendar view across all doctors.
- Add/edit/deactivate doctor and staff accounts.
- Search and view any patient's profile and history.
- Overview of billing status across all patients.
- Manually create, reassign, or cancel appointments on behalf of a patient.

12. Screens / UI Inventory

Screen	Primary Role	Purpose
Login / Signup	All roles	Authentication entry point
Patient Dashboard	Patient	Quick links: book appointment, chat, records
Book Appointment	Patient	Department/doctor + calendar slot picker
My Appointments	Patient	Upcoming and past appointments, reschedule/cancel
Chat Assistant	Patient	Conversational chatbot interface
My Records	Patient	Treatment history and prescriptions
My Bills	Patient	Billing history and dues
Doctor Dashboard	Doctor	Today's schedule, upcoming appointments
Patient History View	Doctor	Read patient history before treatment
Add Treatment Note	Doctor	Diagnosis, procedure, prescription entry
Admin Dashboard	Admin	Clinic-wide calendar and operations overview
Manage Doctors	Admin	Add/edit/deactivate doctor profiles
Manage Billing	Admin	Mark payments, view outstanding dues

13. Suggested Tech Stack

Layer	Technology	Reasoning
Frontend	React.js	Component-based, widely supported, fast to build dashboards with
Backend	Python - FastAPI	Async-friendly, clean integration with LLM SDKs
Database	PostgreSQL	Relational data (patients, doctors, appointments) fits relational schema well
Authentication	JWT	Stateless, role-based access control
Chatbot / LLM	Claude API with function calling	Handles intent understanding and triggers real backend actions
Notifications	Email / SMS gateway	Appointment confirmations and reminders

14. Non-Functional Requirements

- Security: passwords hashed, patient data access restricted by role, HTTPS in production.
- Performance: appointment slot queries should return within acceptable time even as data grows.
- Reliability: no double-booking of the same doctor/slot under concurrent requests.
- Usability: booking flow completable in under 5 user actions.

- Scalability: modular backend so new modules (e.g. payments gateway) can be added later.

15. Cross-Module Edge Cases

- Patient tries to book while another booking for the same slot is mid-transaction (needs slot locking / transaction handling).
- Doctor marked inactive while having future appointments booked - system must flag these for admin reassignment.
- Chatbot attempts an action the patient is not authorised for (must be blocked at the backend, not just the prompt).
- Patient enters ambiguous or incomplete information in chat - chatbot must ask follow-up questions rather than guessing.
- Notification/SMS delivery failure - system should retry or flag for manual follow-up.

16. Build Roadmap (Suggested Phases)

Phase	Scope	Notes
Phase 1	Authentication + Database schema for all modules	Foundation
Phase 2	Patient Management + Doctor/Staff Management	Core data modules
Phase 3	Appointment Management (booking engine)	Core scheduling logic
Phase 4	Treatment Records + Billing	Post-visit workflow
Phase 5	AI Chatbot integration with function calling	AI layer on top of stable backend
Phase 6	Admin Dashboard + notifications + polish	Operations layer and refinement

---

End of Document - Dental Clinic Management System PRD, v1.0